


**Smart Ground-Truthing and Digital Biodiversity System**

# Sprint #1 Report

**Unit Code:** COS30049

**Unit Name:** COMPUTING TECHNOLOGY INNOVATION PROJECT

**Tutor:** Lee Sue Han, Kelvin Yong, Mark Tee, Fu Swee Tee

**Group Name:** Plantiful (Group 7)


| # | Name | Student ID |
|---|---|---|
| 1 | Muhammad Maqeel bin Muhammad Kahfi | 102782384 |
| 2 | Nathan Sebastian Learmonth | 102782258 |
| 3 | Badrul Aliff Aiman bin Badrulmunirzaki | 102778273 |
| 4 | Gae Jayden MWINE | 104393610 |
| 5 | Ashley Wallen Anak Winston | 105806559 |
| 6 | Basill Agas Anak Heatley Rogers | 102778888 |

Executive Summary ..... 4
1. Sprint #1 Plan ..... 4
1.1 What did we plan to do? ..... 4
1.2 Sprint #1 backlog (planned) ..... 4
1.3 Team Responsibility ..... 5
1.4 Expected Output ..... 5
2. Sprint #1 Review Report ..... 6
2.1 What did we achieve? ..... 6
2.2 Backlog Item Status ..... 6
2.3 Definition of Done ..... 7
2.4 Backend and Database Platform (PB-1) ..... 7
2.4 Mobile Offline-First Field Application (PB-2) ..... 8
2.5 Web Knowledge System and Public Visitor View (PB-3, PB-5) ..... 9
2.6 Offline-to-Cloud Synchronisation (PB-4) ..... 10
2.7 IoT Foundational Pipeline (PB-6) ..... 10
2.8 Architecture as Built ..... 11
2.8.1 From Capture to Public Record ..... **Error! Bookmark not defined.**
3. Testing and Quality ..... 16
3.1 Automated Verification ..... 16
3.2 Acceptance Target Status ..... 17
3.3 Security Workstream ..... 17
4. Deviations from the System Proposal ..... 18
5. Sprint Metrics & Risks and Issues ..... 19
5.1 Sprint Metrics ..... 19
5.2 Risks and Issues ..... 19
6. Sprint #1 Retrospective Report (What did we learn and how will we improve?) ..... 20
6.1 What went well ..... 20
6.2 What didn't go well / Problems encountered ..... 21
6.3 Improvements for next sprint ..... 21
6.4 Sprint 2 Plan and Carry-Over ..... 21
7. Sprint #1 Deliverables (What did we actually produced?) ..... 22
7.1 Software ..... 22
7.2 Documentation ..... 22
7.3 Data ..... 23
8. Evidence ..... 23
8.1 Mobile ..... 23
8.2 Web ..... 29

## Executive Summary

Sprint #1 ran from Week 6 to Week 9. It established the working foundation of the Plantiful system. That foundation covers the Supabase backend and database, the offline-first mobile field application, the web Digital Plant Knowledge System with its public visitor view, the offline-to-cloud synchronisation path, and the first end-to-end IoT monitoring pipeline. The sprint goal was to move every major component from design into a runnable state. Doing so reserved Sprint #2 for refinement, security re-assessment, and integration testing rather than first-time construction. That goal was met.

All six product backlog items committed to Sprint #1 were delivered to a demonstrable prototype standard. Those items are PB-1 through PB-6, carrying a combined business value of 51 out of a total 80. The conservation officer review and approval workflow, PB-7, was originally scheduled for Sprint #2. It was delivered ahead of schedule as part of the web knowledge system.

The team verified the developer-facing quality gates that do not depend on a physical device or a live field trial. Mobile and web static type checking pass. Linting passes. The web production build passes. The source-level unit suites for the IoT evaluator, the sync conflict helper, and the acceptance measurement summariser all pass. Security workstream activity ran ahead of the Week 11 gate. Two OWASP ZAP passes were completed with no High-severity alerts outstanding.

The results in this report are deliberately separated into two categories. The first category is checks that were executed and observed. The second category is acceptance measurements that still require a physical device or a live environment. Those measurements are QR lookup timing, offline sync success rate, the five-user usability walkthrough, and physical sensor end-to-end confirmation. This distinction is maintained so that no target in the proposal is reported as met on the strength of a source-only test.

Two implementation decisions taken during the sprint differ from the System Proposal. Both are recorded in Section 6. The first is the synchronisation conflict strategy. It was implemented as revision-checked conflict detection rather than the initial last-write-wins plan. The second is the IoT transport. It was implemented as a USB-first hardware path rather than an MQTT simulation-first path. Both changes were made to strengthen data integrity and to bring real hardware forward. Both are documented with their residual work.

# 1. Sprint #1 Plan

### 1.1 What did we plan to do?

The Initial Release Schedule in the System Proposal Report scoped Sprint #1, covering Weeks 6 to 9, to the six highest business-value backlog items. Those items were standing up the Supabase backend, delivering the first versions of the mobile field app and the web knowledge system, wiring offline-to-cloud sync, publishing a public read-only visitor view, and building the IoT sensor-ingestion and alerting pipeline.

Backlog items 7 through 10 were deliberately deferred to Sprint #2, which covers Weeks 10 to 12. Those deferred items are officer review and approval with reporting, IoT hardware swap-in, full security testing, and final integration.

### 1.2 Sprint #1 backlog (planned)

| # | Backlog item | Depends on | Business value (1–10) |
|---|---|---|---|
| 1 | Set up Supabase project (PostgreSQL schema, auth, RBAC, storage) | N/A | 10 |
| 2 | Deliver mobile offline-first field app: record capture (QR, GPS, camera) | 1 | 9 |
| 3 | Deliver web knowledge system: species record CRUD + search | 1 | 9 |
| 4 | Implement offline-to-cloud sync (SQLite → Supabase) with conflict strategy | 2 | 8 |
| 5 | Add public read-only Visitor view publishing approved records | 3 | 8 |
| 6 | Add IoT sensor ingestion, monitoring dashboard and automated alerts | 4 | 7 |

### 1.3 Team Responsibility

| Name | Role | Sprint #1 backlog items owned |
|---|---|---|
| Badrul Aliff Aiman bin Badrulmunirzaki | Web Knowledge System (Reporting & Maps) | Item 3 (web CRUD/search), Item 5 (visitor view) |
| Nathan Sebastian Learmonth | Web Knowledge System (Records) | Item 3 (web CRUD/search), Item 5 (visitor view) |
| Muhammad Maqeel bin Muhammad Kahfi | Mobile App (Sync & Backend APIs) | Item 1 (Supabase setup), Item 4 (offline sync) |
| Basill Agas Anak Heatley Rogers | Team Lead, Mobile App (Field Data Capture) | Item 2 (mobile capture app) |
| Gae Jayden MWINE | IoT-Based Plant Protection | Item 6 (sensor ingestion, alerts, dashboard) |
| Ashley Wallen Anak Winston | Integration, PM & Documentation | Item 5 (visitor view), cross-team integration |

### 1.4 Expected Output

A working Supabase backend with RLS in place, a mobile app that can capture a record offline and sync it, a web app that can list/search/CRUD species, a public visitor view of approved records, and an IoT pipeline that can turn a simulated sensor reading into an alert, i.e. every layer of the architecture proven end-to-end, even if review/approval and formal security testing were intentionally left for Sprint #2.

# 2. Sprint #1 Review Report

## 2.1 What did we achieve?

The team delivered all six Sprint #1 backlog items and also shipped meaningful progress on item 7 (officially Sprint #2 scope) ahead of schedule. The table below maps each planned item to what actually landed, based on the project's commit history (67 commits across web, mobile, IoT and backend) and the independent verification carried out for the Security Assessment Report.

## 2.2 Backlog Item Status

| Backlog ID | Item | Business value | Status at sprint end |
|---|---|---|---|
| **PB-1** | **Set up Supabase project (PostgreSQL schema, auth, RBAC, storage)** | **10** | **Complete** |
| **PB-2** | **Deliver mobile offline-first field app: record capture (QR, GPS, camera)** | **9** | **Complete** |
| **PB-3** | **Deliver web knowledge system: species record CRUD and search** | **9** | **Complete** |
| **PB-4** | **Implement offline-to-cloud sync (SQLite to Supabase) with defined conflict strategy** | **8** | **Complete** |
| **PB-5** | **Add public read-only Visitor view publishing approved records** | **8** | **Complete** |
| **PB-6** | **Add IoT sensor ingestion, monitoring dashboard and automated alerts** | **7** | **Complete** |
| **PB-7** | **(Pulled forward from Sprint #2) Conservation officer review/approval workflow** | **8** | **Complete ahead of schedule** |

**Committed scope: 6 items, business value 51 of 80. Delivered scope including the pulled-forward review workflow: 7 items, business value 59.**

### 2.3 Definition of Done

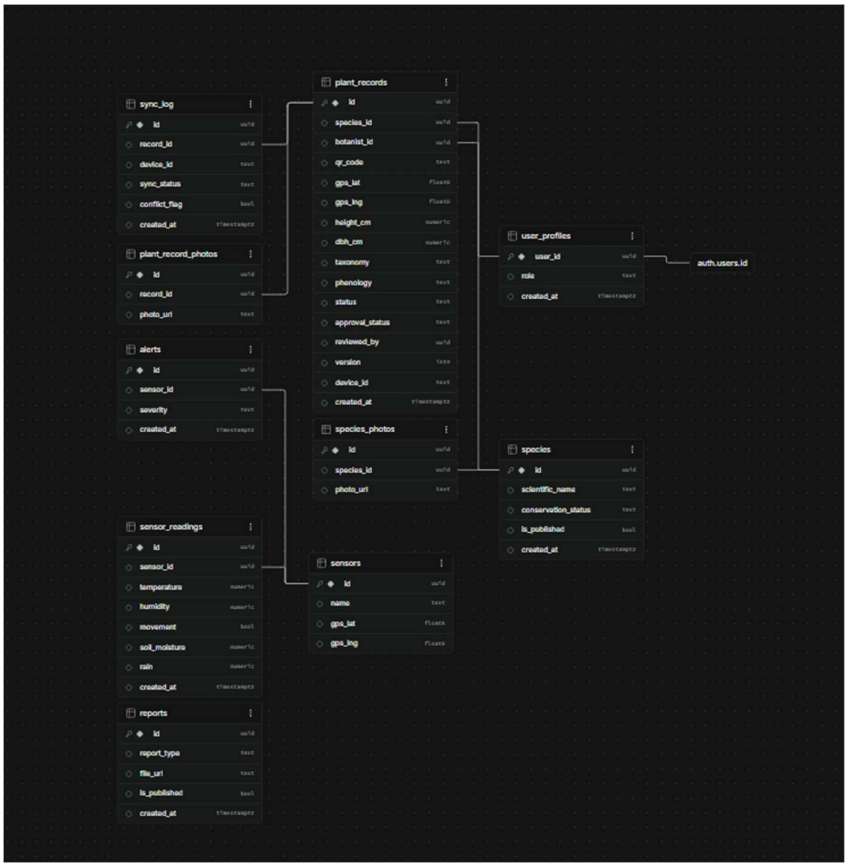
A backlog item was accepted as done only when all of the following held: the feature is implemented in the repository and merged into main through a reviewed pull request; the relevant static checks pass (project type checking and linting, and a successful production build where a build exists); source-level automated tests covering the item's non-UI logic pass where such tests exist; the item can be demonstrated on the built artefacts without manual data patching; and any remaining measured acceptance test that requires a physical device or live environment is explicitly recorded as carry-over, not claimed as complete.

### 2.4 Backend and Database Platform (PB-1)

The Supabase project provides the central database, authentication, role-based access control, and object storage that both client applications and the IoT pipeline depend on.

**Schema.** Ten tables underpin the system: user_profiles, species, species_photos, plant_records, plant_record_photos, sync_log, sensors, sensor_readings, alerts, and reports. Field-record attributes support ground-truthing (taxonomy, phenology, dbh_cm) and the IoT layer adds sensor_readings.soil_moisture and sensor_readings.rain. **Migrations.** The migration history was consolidated into two idempotent scripts, 001_schema.sql for a fresh project and apply_project.sql for everything else (row-level security policies, schema backfills, workflow triggers, photo visibility, the approval gate, and storage-bucket policies). The per-feature files are retained under applied/ for reference only.

**Security at the data layer.** Row-level security enforces the role model for botanist, conservation_officer, admin, and anonymous visitors. Object storage uses three buckets (record-photos, species-photos, and a deliberately private reports bucket). A reviewer-stamping trigger clears prior review state when an owner resubmits an edited record. **Verification.** The backend verification scripts report the anonymous write-block property as passed at the database layer, and the record photo write path as confirmed.

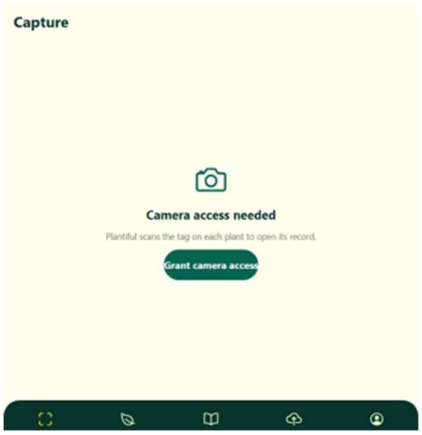


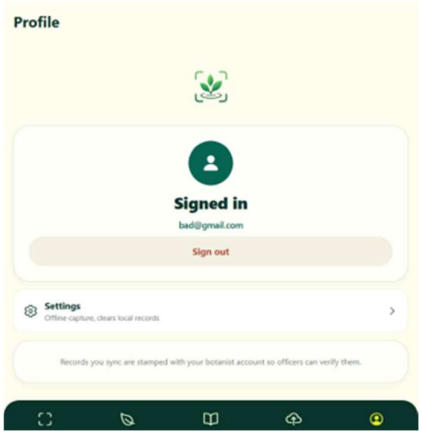

## 2.4 Mobile Offline-First Field Application (PB-2)

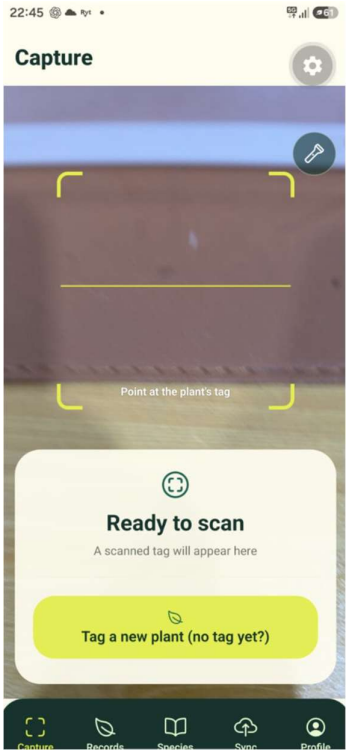
The mobile application is built with React Native 0.86 on Expo SDK 57. It is scoped to the botanist role.

A capture flow records the species, height, morphology, and ground-truthing attributes. It attaches camera photos, stamps GPS, and links the record to its QR tag. A scan flow reads the plant tag and resolves the record against a locally cached copy when offline. The scan also records its own lookup timing as an acceptance measurement. A local SQLite store at schema version 9 holds records, photos, and sync state. This lets capture work with no connectivity.

A records view lists captures and their sync state. A profile view manages the signed-in botanist and role. Capture routes are guarded so that only the botanist role can create records. Other roles get read-only behaviour.




### 2.5 Web Knowledge System and Public Visitor View (PB-3, PB-5)

The web application is built with Next.js 16.3.6, React 19.2.8, and Tailwind CSS 4. It serves conservation officers, administrators, and the public.

The records list and detail views present synced captures with species search. Officers can approve or reject submissions. The approvals page provides a focused pending queue. Species management supports create, edit, and publish. The public catalogue is limited to approved and published data.

The system also includes a map view for located observations. It includes a reports view with CSV and PDF export. It includes an admin users page for role grants and revocations. It includes account pages.

The /explore route is a read-only public portal. It lists only approved and published records and requires no account. All write attempts are blocked by row-level security at the database layer rather than in the user interface alone.

The production build compiled nineteen routes. The visitor-view write-block was verified at the database layer.

### 2.6 Offline-to-Cloud Synchronisation (PB-4)

Synchronisation moves records and their photos from the local SQLite store to Supabase once connectivity returns.

Synchronisation moves records and their photos from the local SQLite store to Supabase once connectivity returns.

The sync centre pushes pending records along with their photos. It marks each record synced only after the server acknowledges it.

Same-owner edits made on two devices are protected by a revision check. Each record carries a version field and updates use a compare-and-swap on that version. A stale or unknown revision is surfaced as a visible conflict status. The local edit is preserved and the owner can retry explicitly. Cross-owner edits are blocked by row-level security.

Each push attempt is recorded locally in an acceptance_measurements table. This lets a sync success rate be calculated from real device data.

This strategy delivers the manual conflict queue that the System Proposal listed as an enhancement. It does not rely on last-write-wins for same-owner edits. This is recorded as a deviation in Section 6.

### 2.7 IoT Foundational Pipeline (PB-6)

The IoT layer ingests environmental readings and raises automated alerts.

An ESP32 node carries a DHT11 temperature and humidity sensor, a PIR motion sensor, a soil-moisture probe, and a rain sensor. It produces readings and emits a structured line per reading over serial. A host bridge reads the serial stream, registers the sensor, and inserts each reading into sensor_readings. The bridge then evaluates the reading for alerts immediately.

The evaluator raises three alert types. heat_stress has high and critical thresholds. drought_stress is driven by low air humidity. unusual_movement covers motion. A cooldown suppresses repeated alerts for the same condition. A periodic pass detects silent sensors.

The officer dashboard at /alerts presents per-sensor status, environmental trend charts, movement and rain activity, and an alert timeline. It has a range selector and automatic refresh.

During the sprint, live readings were inserted roughly every two seconds. unusual_movement, heat_stress, and drought_stress alerts were created and observed on the dashboard.

The physical end-to-end path is proven to the point of a real node producing real readings and real alerts in the database and on the dashboard. Longer-duration field confirmation remains a carry-over item.

## 2.8 Architecture as Built

The implemented architecture matches the three-tier design in System Proposal Section 6.2.

**Client tier.** A React Native offline-first mobile application for botanists and a Next.js web application for officers, administrators, and public visitors. **Application and data tier.** Supabase provides PostgreSQL, authentication, row-level security, the auto-generated REST interface, and object storage. The IoT ingest bridge is a small host-side service that uses the service-role key from host environment variables only. **Transport.** REST connects the mobile store, the web application, and the IoT bridge to Supabase. MQTT is retained as future work and is not required by the current path. **Offline store.** SQLite on the device holds captures and their photos until synchronisation succeeds.

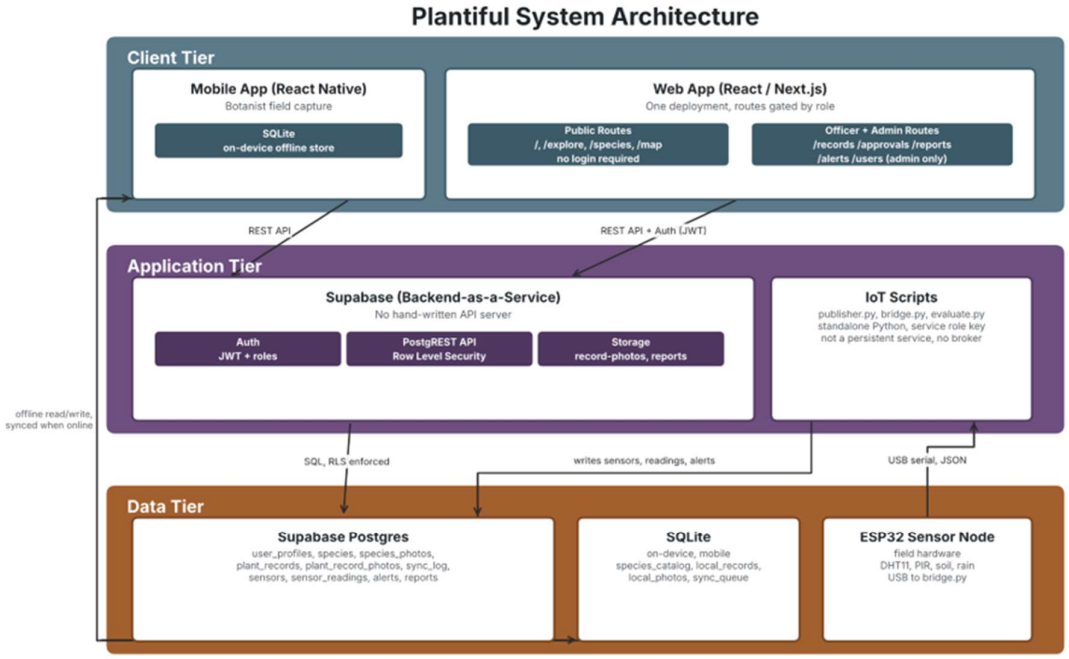



*Figure 1. Plantiful system architecture: client, application, and data tiers (current codebase, origin/main).*

The IoT pipeline writes straight to Supabase over PostgREST with a service role key; there is no MQTT broker.

Public, officer, and admin routes all live in the one Next.js app; access is a server-side check per route.

# 3. How the System Works

This section explains every part of the delivered system in plain language. It is written for a reviewer who has not read the source code. Each part is described by what it does, who uses it, and how it communicates with the rest of the system.

## 3.1 The Three-Tier Structure

The system is built as three connected tiers. The client tier holds the two applications people interact with, which are the mobile field app and the web knowledge system. The application and data tier is a single cloud backend that stores every record, enforces every permission, and hosts the sign-in service and file storage. The device tier is the sensor node that monitors a plant's surroundings and reports what it measures.

All three tiers meet at the same backend. There is one database, one set of user accounts, and one permission model. A record created on a phone in the forest becomes visible to an officer on a laptop within seconds of connectivity returning. A sensor reading taken at a plant's location appears on the same officer dashboard that shows field records. Nothing is duplicated between separate stores.

## 3.2 The Mobile Field Application

The mobile application is the botanist's tool in the field. It is installed on a phone or tablet and works with or without a network connection. Its purpose is to capture a complete, structured observation of a plant and hold it safely until it can be sent to the central database.

**Signing in.** The botanist signs in with an email address and password. The application remembers the session, so daily use does not require repeated sign-in. Only accounts with the botanist role can create new records. Other roles open the application in a read-only state, which prevents accidental data entry by the wrong person.

**Finding the species.** Before capturing, the botanist can browse or search the species catalogue. The catalogue is cached on the device, so it remains searchable in areas with no signal. This supports identification in the field and reduces the chance of recording the wrong species name.

**Capturing an observation.** The capture flow walks the botanist through each required piece of information. It starts with the species selection, then collects the taxonomy, the morphological characteristics, the plant height, and any other attributes the project requires. The form prevents submission until the required fields are filled, which protects data quality at the point of entry rather than after the fact.

**Photographs.** The botanist photographs the plant using the device camera. Photographs are attached directly to the observation, so evidence and data stay together. Images are stored on the device first and uploaded later during synchronisation.

**Location.** The application stamps the device's GPS position onto the observation automatically. This removes the need for manual coordinate entry and gives every record a verifiable location.

**Plant tags and QR codes.** Each plant record generates a QR code. A printed tag carrying that code is fixed at the plant in the field. Scanning the tag with the application opens the stored record

immediately, which lets a returning botanist see what was previously recorded at that exact plant without searching the catalogue. Scanning also works against the local cache when offline.

**The offline store.** Everything the botanist captures is written to a small database on the device. This local store holds records, their photographs, and their synchronisation state. Because the source of truth during fieldwork is the device itself, the botanist can work deep in the forest for a full day with no connectivity and lose nothing.

**Sending data to the cloud.** When connectivity returns, the botanist opens the synchronisation centre and sends the pending items. Each record is marked as sent only after the central database confirms receipt. Anything that fails stays visible in the queue with its reason, so nothing disappears silently.

**Protecting edits made in two places.** If the same record is edited on two devices, the system detects the clash rather than overwriting one version with the other. Each record carries a version marker. When an edit arrives that does not match the current version in the cloud, the application holds it as a conflict, keeps the local copy intact, and shows it to the owner for an explicit decision. This prevents silent data loss.

**Measuring reliability.** Every send attempt is recorded on the device with its outcome. These measurements are exported after trials so the team can report a real sync success rate rather than an estimated one.

**The profile area.** The profile screen shows who is signed in and under which role. It also lets the botanist manage their own account details.

### 3.3 The Web Knowledge System

The web application serves three audiences, which are conservation officers, administrators, and the general public. It runs in an ordinary browser on a computer or tablet and is the management and publication face of the system.

**Signing in.** Officers and administrators sign in with email and password. The public visitor portal requires no account at all, because it only ever shows content that has already been approved for release.

**The records workspace.** Field observations that have synced from mobile devices appear here. Officers can search the list by species, open a record to see its full detail, review its photographs, and check its location. This is where ground-truthed data is inspected before it is trusted.

**The approvals queue.** New submissions arrive in a focused pending queue. An officer reviews each one and either approves or rejects it with a reason. The approval decision, the officer who made it, and the time it was made are stamped onto the record permanently. This creates an audit trail and satisfies the project requirement for review and approval of submitted observations.

**Species management.** Officers can add, edit, and delete species entries. Each species entry holds the taxonomy, the scientific and common names, the description, the ecological information, the conservation status, and any cultural significance notes. Officers also upload botanical photographs and mark a species as published. Publishing is blocked until an approved field record exists for that species, which stops empty or unverified entries from reaching the public.

**Search and filtering.** The catalogue can be searched and filtered by scientific name or common name. This directly serves the project requirement for retrieval of biodiversity information by researchers, officers, and guides.

**The public visitor portal.** This is a read-only section of the site that lists only approved and published records and species. It needs no login. It is the answer to the second problem statement, which is that visitors and communities currently have poor access to up-to-date biodiversity information. Anyone with a browser can now browse verified plant knowledge.

**The distribution map.** Located observations are plotted on a map. Officers and the public can see where each species has been recorded, which supports the requirement for species distribution maps and helps conservation planning.

**Report generation.** Officers generate biodiversity reports from the records and species data and export them in standard document formats. The exports are stored in a private area that only officers can reach, because reports may contain the locations of sensitive species.

**The administrator's user page.** Administrators can grant and revoke roles from this page. This is how a new officer or botanist is onboarded, and how access is removed when a person leaves a project.

**The IoT monitoring dashboard.** This page is reserved for officers and administrators. It shows the live state of every registered sensor, the environmental trend charts, the movement and rain timelines, and the list of raised alerts. It has a time-range selector so an officer can inspect the last day, the last five days, the last week, or the entire history. Live readings and alerts refresh automatically every ten seconds. Chart history refreshes every minute. A manual refresh button is available for immediate checks.

### 3.4 The IoT Monitoring Chain

The IoT chain exists to watch the surroundings of a protected plant continuously and to raise an alert when something threatens it. It has four stages, which are sensing, transport, evaluation, and display.

**Sensing at the plant.** A small microcontroller board is deployed near the plant. It carries four sensors. A temperature and humidity sensor measures the air. A motion sensor detects movement near the plant. A soil moisture probe measures the water level in the ground. A rain sensor detects whether it is raining. The board also carries a GPS receiver, so every reading is tied to a physical location. Together these satisfy the requirement for real-time collection of location, temperature, humidity, and movement data.

**Transporting the readings.** The board produces a structured report roughly every two seconds and sends it over a USB cable to a host computer. Using a cable rather than a wireless link was a deliberate sprint decision, described in Section 6. It brought real hardware forward and removed the reliability risk of an untested wireless path during the first build.

**The host bridge.** A small program on the host computer reads each incoming report, registers the sensor in the central database on first contact, and stores every reading. The bridge holds the privileged database credential on that host only. The credential never appears in the board's firmware, the mobile application, or the web client.

**Evaluating each reading.** Immediately after a reading is stored, the system checks it against a set of rules. High and critical heat thresholds raise a heat stress alert. Low air humidity raises a drought

stress alert. Motion raises an unusual movement alert. A cooldown period stops the same condition from raising the same alert repeatedly. A separate periodic check notices when a sensor has stopped reporting and raises a silent sensor alert. These rules are the project's automated threat detection.

**The officer dashboard.** Raised alerts and the latest reading for each sensor appear on the officer dashboard. An officer can see at a glance which sensors are healthy, what the current conditions are, what has been detected, and when. This is the centralized monitoring dashboard and rapid response capability the project requires. Every reading and every alert is stored permanently, which also provides the long-term ecological data record for later habitat analysis.

### 3.5 The Database and Security Model

All data lives in one cloud database. Its job is to hold the information consistently and to decide, for every request, whether the person or device asking is allowed to see or change it.

**What is stored.** The database holds user accounts and their roles, the species catalogue and its photographs, field observations and their photographs, synchronisation bookkeeping, sensors and their readings, raised alerts, and officer report exports.

**Roles.** There are three privileged roles and one public role. The botanist captures field data. The conservation officer reviews, approves, publishes, generates reports, and monitors IoT alerts. The administrator manages users and roles. The public role sees only approved and published content.

**Enforcing permissions inside the database.** Every table carries a set of rules that run inside the database engine itself. These rules decide, per role and per action, whether a request is allowed. A botanist can create and edit only their own observations. An officer can read and act on everything that needs review. A member of the public can read only approved and published rows. Because the rules live in the database and not in the application's interface, they hold even if someone bypasses the application entirely and talks to the database directly.

**File storage.** Photographs and report exports are held in dedicated storage areas with their own permission rules. Field photographs and species photographs are readable by the roles that need them. Officer report exports sit in a deliberately private area, because they may contain the coordinates of sensitive species.

**The approval gate.** A species cannot be published to the public catalogue until it has at least one approved field record. This gate is enforced by a rule inside the database, so there is no path by which an unverified species reaches the public.

**Transport security.** Every connection between a client and the backend is encrypted in transit by the platform. Sign-in sessions are issued and verified by the backend's authentication service, not by any client-side logic.

### 3.6 How a Single Observation Travels Through the System

The following narrative ties every part together.

A botanist signs into the mobile application at the start of a survey. She scans the QR tag on a plant she has visited before, and the stored record opens from the device's local cache. She adds a new photograph and updates the plant's height. The application stamps her current GPS position and saves everything to the local store. No network is available, and none is needed.

At the end of the day she reaches an area with signal and opens the synchronisation centre. The application sends the updated record and its photograph. The database checks that she is the owner of the record and that her version marker matches, then accepts the change and confirms receipt. The record is marked as synced on her device.

Back at the office, an officer opens the approvals queue and sees the updated record. She reviews the photograph, confirms the species identification, and approves the change. The approval, her identity, and the time are stamped onto the record. The species remains published, and the updated photograph now appears on the public visitor portal for anyone to see.

Meanwhile, near the plant itself, the sensor board has been taking readings every two seconds all day. The host bridge has stored each one. A rise in temperature has crossed the heat threshold, so a heat stress alert was raised and appears on the officer dashboard. The same officer who approved the record sees the alert, notes the location, and can plan a site visit.

This is the complete loop the project set out to build. It runs from a manual, paper-based process to a digital, verifiable, and monitored one.

# 3. Testing and Quality

## 3.1 Automated Verification

The following checks were executed and observed during the sprint. The scope column records exactly what each result does and does not cover.

| Area | Command | Result | Scope |
|---|---|---|---|
| Mobile typing | npx tsc --noEmit | **Pass** | Static type checking |
| Mobile lint | npx expo lint | **Pass** | Source linting |
| Revision-checked edit | npm run test:coverage | **Pass, 3 tests; 100% coverage of sync-edit.ts only** | Stale update, unknown revision, server error |
| Measurement summariser | python -m unittest discover -s mobile/scripts | **Pass, 2 tests** | Retries, denominators, zero samples |
| IoT bridge and evaluator | python -m unittest discover -s iot | **Pass, 7 tests** | Timestamp handling, ingestion, immediate alert, cooldown, invalid node IDs, silent-sensor detection, severity |

| | | | |
|---|---|---|---|
| Web typing | npx tsc --noEmit | **Pass** | Static type checking |
| Web lint | npm run lint | **Pass, one pre-existing image optimisation warning** | Source linting |
| Web production build | npm run build | **Pass, 19 routes compiled** | Compilation only, no live request verification |

### 3.2 Acceptance Target Status

The following table reports each measurable target from System Proposal Section 7.0 against its status at the end of Sprint #1. Targets are marked as verified only where evidence exists.

| Quality attribute | Target | Status at sprint end |
|---|---|---|
| **Reliability (offline sync)** | More than 95% of test submissions sync without data loss | Implementation complete; measured trial pending |
| **Functional performance** | QR scan to record in under five seconds average | Instrumentation built; measured trial pending |
| **Security** | No critical or high vulnerabilities remaining after remediation | Two ZAP passes complete with no High findings; final re-assessment pending |
| **Code quality** | More than 80% coverage on backend API and mobile sync module | Pending; helper currently at 100% of sync-edit.ts only |
| **Usability** | More than 90% unassisted task completion across a five-user walkthrough | Pending; walkthrough not yet run |
| **Data integrity** | 100% of required fields validated before sync | Validation present; backend validation testing pending |
| **Public access correctness** | 100% of unauthorised writes fail with no data change; unapproved records never visible | **Verified at the database layer** |

### 3.3 Security Workstream

Security activity ran ahead of the Week 11 gate. The security assessment progressed through static analysis, dependency scanning, row-level security verification, a manual secure-code review, and two dynamic passes with OWASP ZAP 2.17.0 (an anonymous baseline and an authenticated officer

session). Twelve findings were recorded across the assessment, and the remediated ones were confirmed by re-scan. No High-severity ZAP alerts were outstanding at the end of the sprint. The full detail is in Docs/Reports/Security_Assessment_Report.md with evidence under Docs/Reports/evidence/.

| Check | Scale | Result |
|---|---|---|
| Web production dependencies | 27 packages | 0 vulnerabilities |
| Mobile production dependencies | 588 packages | 26 advisories, all build-toolchain only (0 critical, 16 high, 10 moderate), no runtime reach |
| Database-layer RLS / write-block verification | 10 checks | 10 of 10 PASS |
| OWASP ZAP dynamic scan (anonymous + authenticated) | 2 passes, 10 Oct 2026 | 0 High-severity alerts |
| Findings register (Security Assessment Report v1.1) | 12 findings + 2 observations | 11 fixed, 1 mitigation defined (F-06, no runtime reach), 0 confirmed critical/high residual |

## 4. Deviations from the System Proposal

The following differences between the System Proposal and the delivered prototype are recorded here so that design intent can be distinguished from delivered behaviour.

| Area | System Proposal | As built in Sprint #1 | Reason | Remaining work |
|---|---|---|---|---|
| Sync conflict strategy | Last-write-wins first, with a manual conflict queue as an enhancement | Revision-checked compare-and-swap on plant_records.version, with a visible conflict status and manual resolution | Stronger data integrity; protects same-owner multi-device edits without silent overwrite | None for the core path; the two-device conflict evidence trial is still to be run |
| IoT transport | Simulated sensors over MQTT first, with the ESP32 as a later swap-in | USB-first path with a real ESP32 node, a host bridge, and a periodic evaluator | Real hardware was available, so the true pipeline was built directly | MQTT remains future work; longer-duration physical confirmation is pending |

| | | | | |
|---|---|---|---|---|
| Officer review and approval (PB-7) | Scheduled for Sprint #2 | Delivered in Sprint #1 with the web knowledge system | The web system and the approval gate share one data model, so building them together reduced rework | Final integration testing remains in Sprint #2 |
| Sensitive-data encryption | Objective refers to encryption of personal records and reviews | Confidentiality currently enforced through authentication and row-level security, with TLS in transit | Encryption posture is assessed in the security report | Field-level encryption is to be confirmed against the stated objective before submission |
| Optional innovation (Objective 5) | Proposed, pending tutor confirmation | Design write-up complete; prototype not built | Documented as optional in the proposal | Pursued only with tutor agreement |

## 5. Sprint Metrics & Risks and Issues

### 5.1 Sprint Metrics

| Metric | Value | Note |
|---|---|---|
| **Committed backlog items** | **6** | PB-1 to PB-6 |
| **Completed backlog items** | **6, plus PB-7 pulled forward** | 7 total delivered |
| **Committed business value** | **51 of 80** | Weighted by the proposal's business value column |
| **Delivered business value** | **59 of 80** | Includes the pulled-forward review workflow |
| **Automated unit tests passing** | **12** | IoT 7, mobile sync helper 3, summariser 2 |
| **Web routes compiled** | **19** | Production build |
| **Open High-severity security findings** | **0** | After two ZAP passes |

### 5.2 Risks and Issues

The Sprint #1 risk posture against the register in System Proposal Section 7.1, with the issues actually encountered.

| Ref | Risk or issue | Status at sprint end | Action |
|---|---|---|---|
| **R1** | Data loss during offline-to-cloud sync | Mitigated in design | Revision check and preserved local copy; measured sync trial pending |
| **R2** | IoT hardware unavailable within the trimester | Retired | A real ESP32 node was obtained and used |
| **R3** | Poor or no connectivity in the field | Mitigated in design | Offline store and deferred sync implemented |
| **R4** | Security vulnerabilities discovered late | Partially mitigated | Two ZAP passes complete; final re-assessment pending |
| **R5** | Scope creep against the timeline | Managed | Review workflow pulled forward deliberately; optional innovation held back |
| **R6** | Duplicate or inconsistent species records | Mitigated in design | Stable QR record ID and mandatory-field validation |
| **R7** | Publicly exposing unapproved data | Mitigated and verified | Read-only route group and row-level security; write-block verified |
| **I1** | Sensor readings unstable during wiring changes | **Open** | DHT11 reported an out-of-range reading and the soil probe fluctuated when disturbed; wiring must be reseated and analog thresholds calibrated before final field evidence |
| **I2** | Development credentials exposed during setup | **Open** | Service-role key and test-account passwords must be rotated before submission |

# 6. Sprint #1 Retrospective Report (What did we learn and how will we improve?)

## 6.1 What went well

The backend-first sequencing worked: building Supabase first meant the mobile, web, and IoT workstreams could run in parallel without blocking each other. A single TypeScript stack across

mobile and web let members move between workstreams and reduced duplicated effort. The security workstream ran ahead of its gate, which leaves remediation time before the final demo. The extras in the mobile sync path, conflict detection and the acceptance measurement instrumentation were delivered without slipping the committed scope.

### 6.2 What didn't go well / Problems encountered

Git coordination broke down on a few shared files. Work on the officer auth-gate and the records-browser status/conservation filters was done on a branch that went three commits without syncing with main, while main itself moved forward by over sixty commits, including three separate changes to that same records-browser file. Reconciling the two later produced a five-file merge conflict (records-browser.tsx, map/page.tsx, records/page.tsx, signin/page.tsx, auth.ts) that was costly to untangle, and part of the local work ultimately had to be discarded and redone against the current main rather than merged.

Three real application defects (F-01, F-02, F-03) were only caught during the formal security review rather than during normal development, notably F-01, where every mobile photo upload was silently failing for an unknown period before it was found.

The IoT pipeline shipped a simplified transport (direct Supabase writes) instead of the MQTT plus InfluxDB pipeline specified in the proposal. Not inherently a problem, but it means Sprint #2's "IoT alert threshold tuning and ESP32 hardware swap-in" (backlog item 8) needs re-scoping against what was actually built, not the original design.

Physical sensor wiring was less stable than expected, and a disturbed DHT11 produced an out-of-range reading that triggered spurious alerts; analog soil and rain thresholds are still calibration-dependent. Development credentials were handled loosely at setup time, which created a cleanup task that should not have been necessary. Some acceptance measurements were only instrumented during the sprint, so the measured trials were deferred to Sprint #2 rather than completed alongside the features.

Photo coverage is inconsistent. At the species level, roughly 78 of 263 species (about 30%) still have no photo, going by the 185 species_photos rows against 263 species. At the field-record level coverage looks healthier (14 photos across 11 records). The species-level gap is most likely a sourcing limit rather than an upload failure, since the iNaturalist import only pulls a photo when it finds one under an acceptable open license, but that's worth the team confirming rather than assuming.

### 6.3 Improvements for next sprint

Pull from main daily, or before starting any change on a shared file, to avoid long-lived branch divergence. The merge conflict this sprint came directly from going too long without syncing.

The team agreed three concrete changes for Sprint #2: add a short hardware bench check with a warm-up and a calibration step before any field evidence is captured; use a single documented secrets workflow, with no keys in source, in chat, or in client bundles; and run the measured acceptance trials (QR timing, sync success, five-user walkthrough) as scheduled work items rather than as optional follow-ups.

### 6.4 Sprint 2 Plan and Carry-Over

Sprint #2 covers the remaining backlog items and the integration, security, and documentation close-out.

| Backlog ID | Item | Focus for Sprint #2 |
|---|---|---|
| **PB-7** | Conservation officer review/approval workflow | Delivered early; final integration testing only |
| **PB-8** | IoT alert threshold tuning and ESP32 hardware swap-in | Calibrate thresholds against observed conditions and confirm the hardware path end to end |
| **PB-9** | Complete security: vulnerability scan, remediation, re-verify | Final ZAP pass, confirm zero critical and high findings, rotate exposed credentials |
| **PB-10** | Final integration, system testing, documentation and demo | Full round-trip testing, user manual, final report, and live demo |

**Carry-over items from Sprint #1 that must be closed in Sprint #2.** Run the QR scan-to-record timing trial on a physical device using the acceptance measurement export; run the offline capture and idempotent push trial and report the sync success rate; run the two-device same-owner conflict trial and record the conflict evidence; run the physical sensor trial with a calibrated node and confirm one reading and its alert end to end; conduct the five-user usability walkthrough against the draft user manual; and rotate the exposed service-role key and reset the test-account passwords.

# 7. Sprint #1 Deliverables (What did we actually produced?)

## 7.1 Software

- **Web application** (Next.js + Supabase): species catalog, filterable records browser, an officer approval workflow (ahead of schedule), interactive species map, report generator, role-based access control, and a public visitor explore page.
- **Mobile application** (Expo / React Native): offline-first field capture with photo, GPS and plant-tag support, background sync with duplicate handling, species reference browsing, and record editing.
- **IoT simulation:** a simulated sensor-feed publisher and a rule-based threat evaluator, writing to the shared database via a PostgREST client (simplified transport, see Section 2.9 scope note).
- **Database:** a Supabase/Postgres schema (species, plant_records, species_photos, user_profiles, sensors, sensor_readings, alerts, reports) with Row-Level Security enforced on every application table and independently verified.

## 7.2 Documentation

- **Security Assessment Report** (Docs/Reports/Security_Assessment_Report.md, v1.1): a full SSDLC-aligned assessment covering static analysis, dependency scanning, row-level security verification, a manual secure-code review, and two OWASP ZAP dynamic passes, closed out with a twelve-item findings register.
- **RLS verification script** (backend/scripts/verify_public_write_block.sql): a repeatable, idempotent SQL test proving the public write-block guarantee; all 10 checks PASS.

- **System Proposal Report** (v1.6): the project's full design and planning document, covering the problem statement, solution analysis, architecture, risk register, and the Sprint #1/#2 backlog this report is measured against.
- **Component READMEs** for the web app, mobile app, backend and IoT service, each covering setup and key decisions.

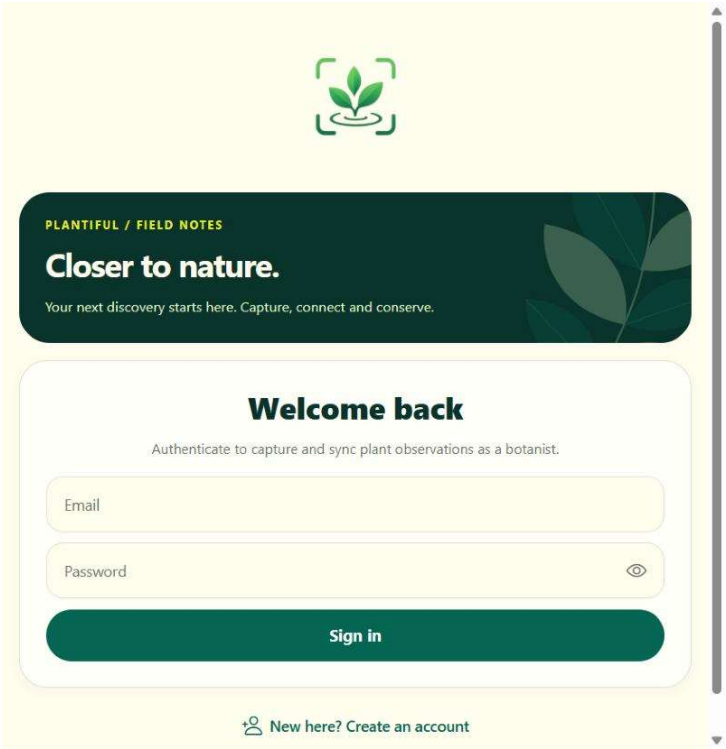
### 7.3 Data

- **Species catalogue:** 263 plant species in the database, covering taxonomy, conservation status and descriptions (see the species catalogue screenshot in Section 8.2).
- **Species photos:** 185 species_photos rows linked to the catalogue, sourced via the iNaturalist import tooling with attribution tracking. Roughly 78 of the 263 species (about 30%) still have no photo; this gap is most likely a sourcing limit rather than an upload failure, since the import only pulls a photo when one is available under an acceptable open licence (see Section 6.2).
- **Field records:** 11 field observations synced from the mobile app, with 14 photos across those records.
- **Public visitor view:** a filtered, approved-and-published projection of the catalogue powers the /explore visitor portal.

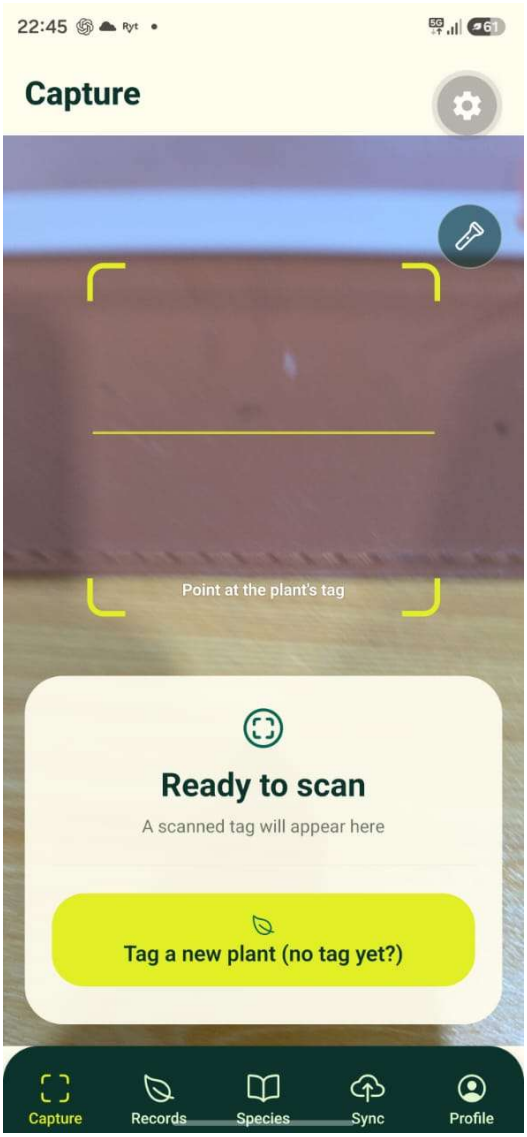
## 8. Evidence

### 8.1 Mobile

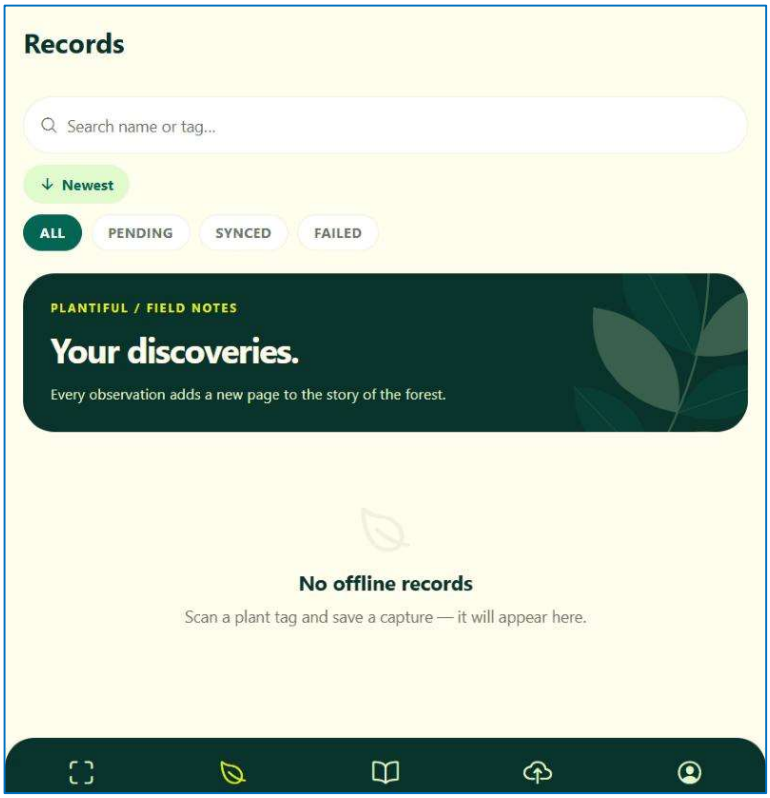
1. Sign in



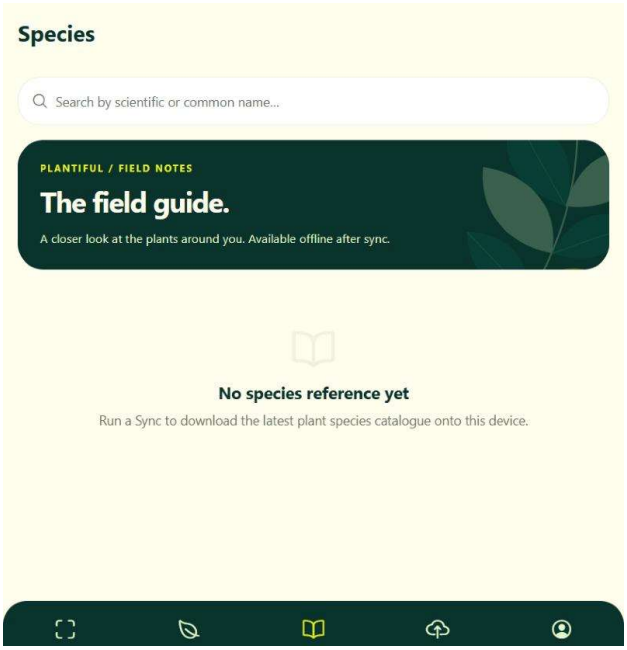
2. Capture Camera



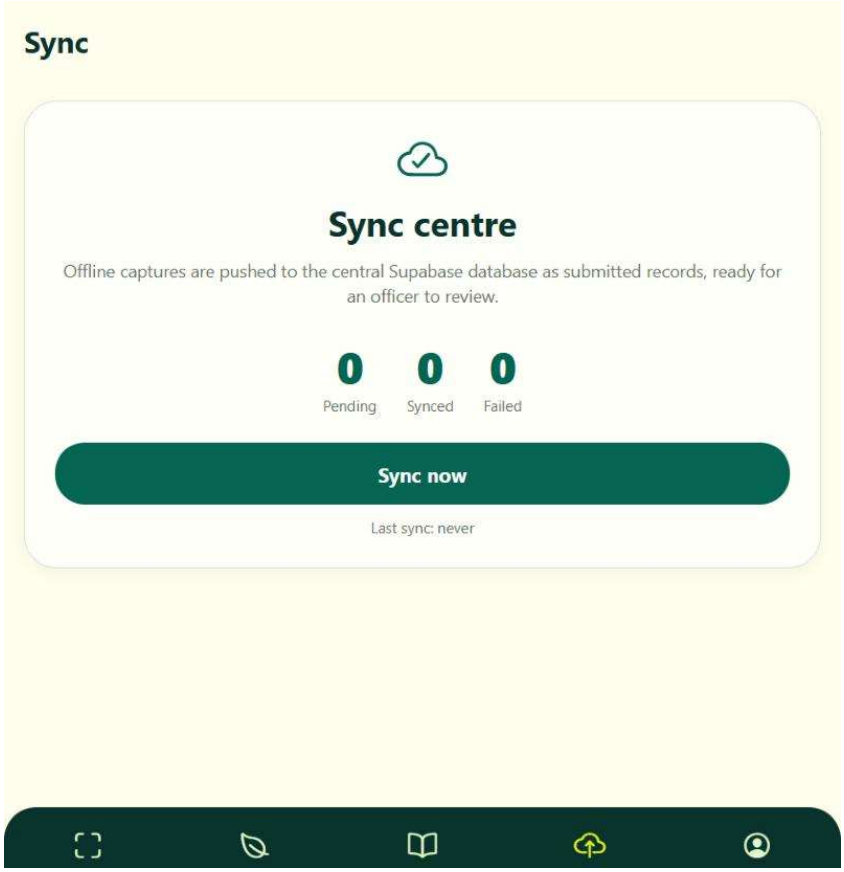
3. Records empty



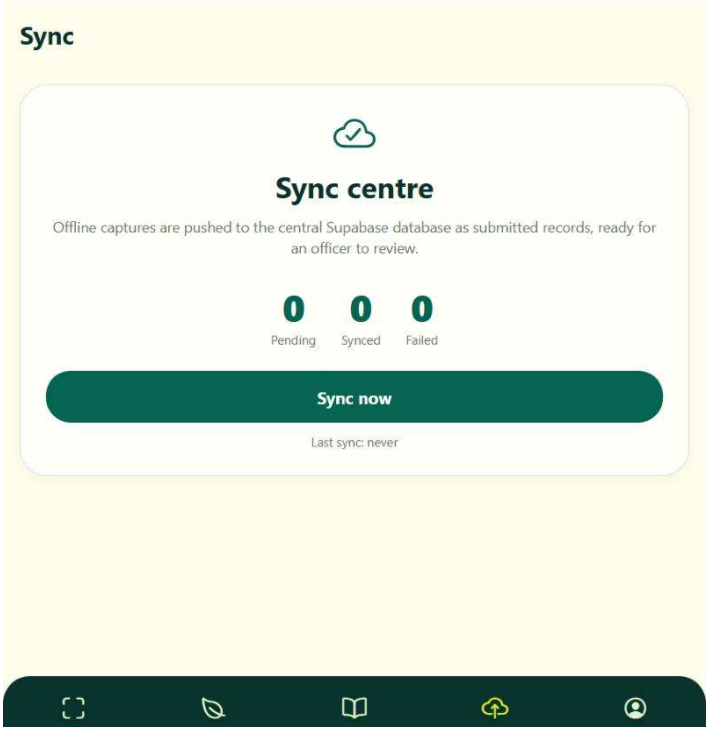
4. Species Empty



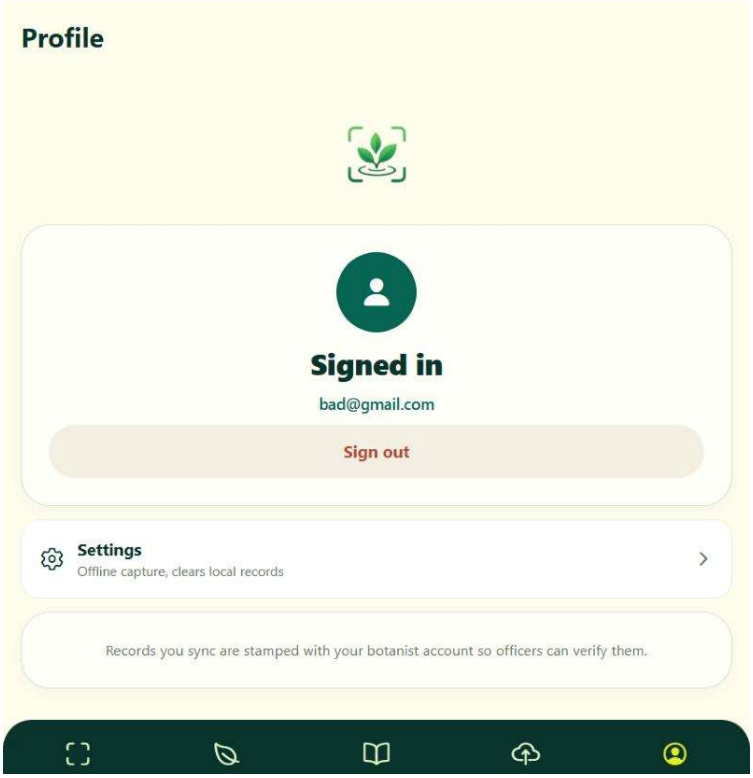

5. Sync Center



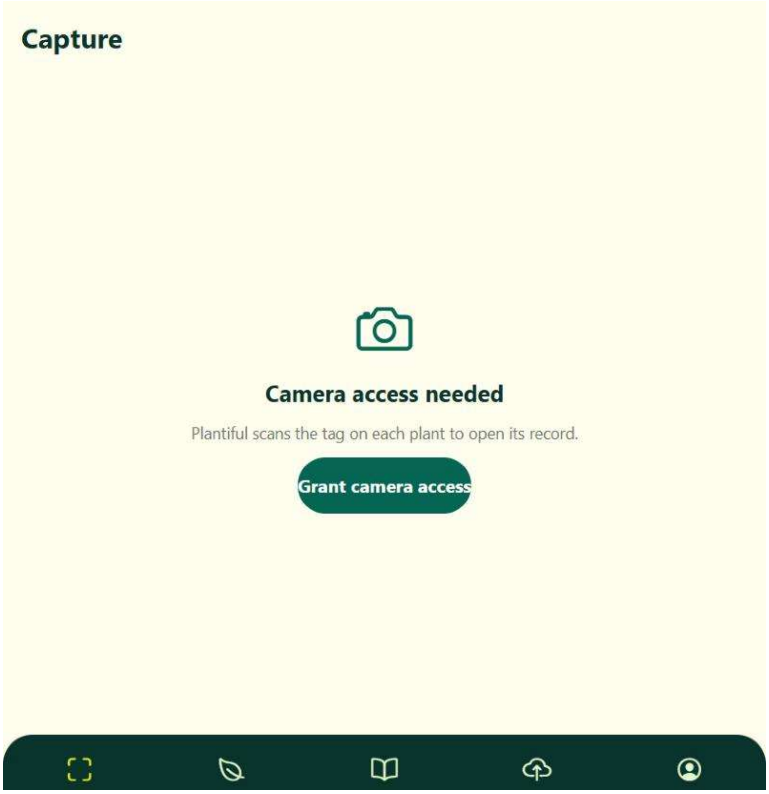

6. Sync Attempted


Sync
Sync centre
Offline captures are pushed to the central Supabase database as submitted records, ready for an officer to review.
0
Pending
0
Synced
0
Failed
Sync now
Last sync: never

7. Profile

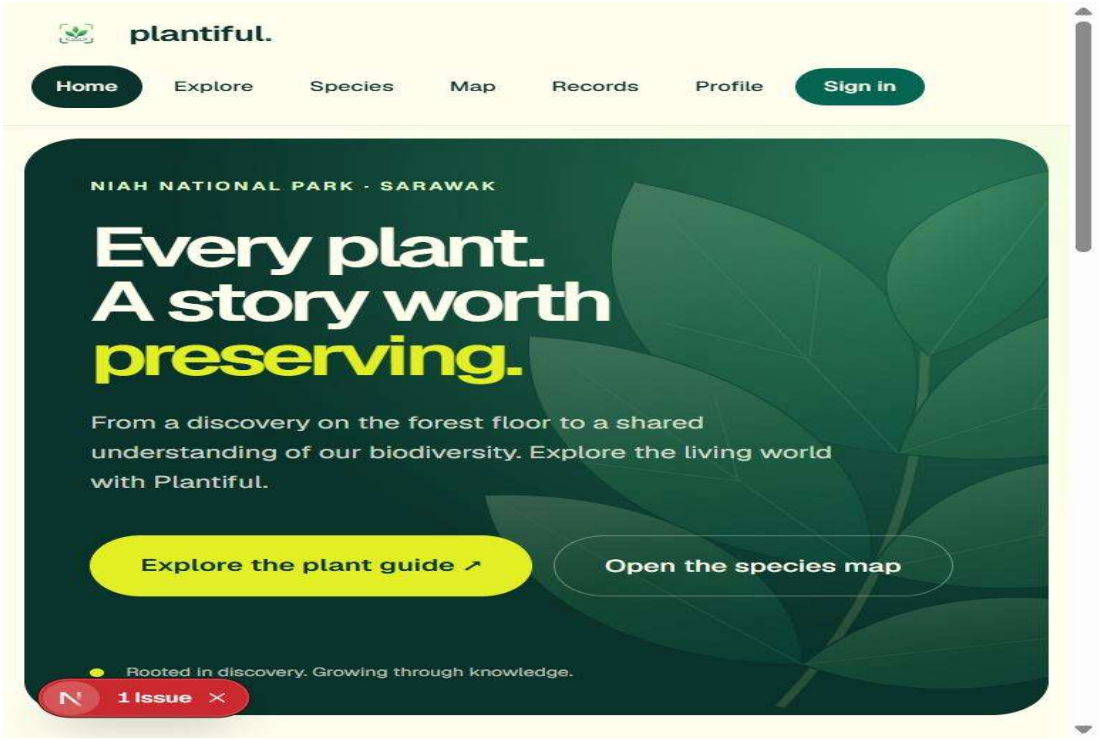



8. Capture Visit

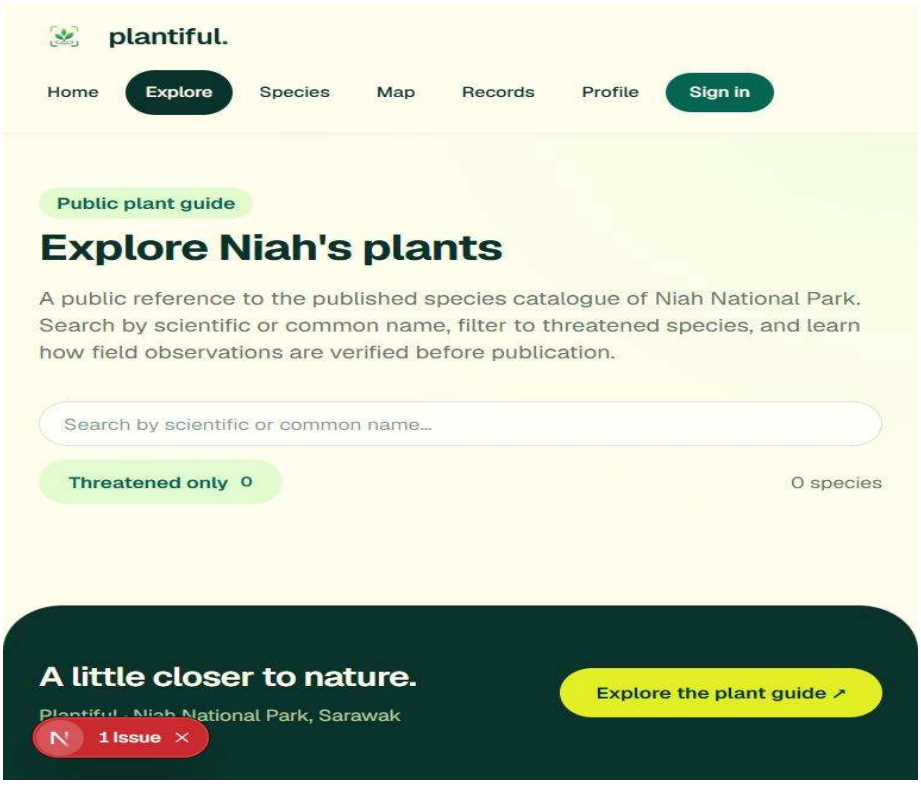



**8.2 Web**

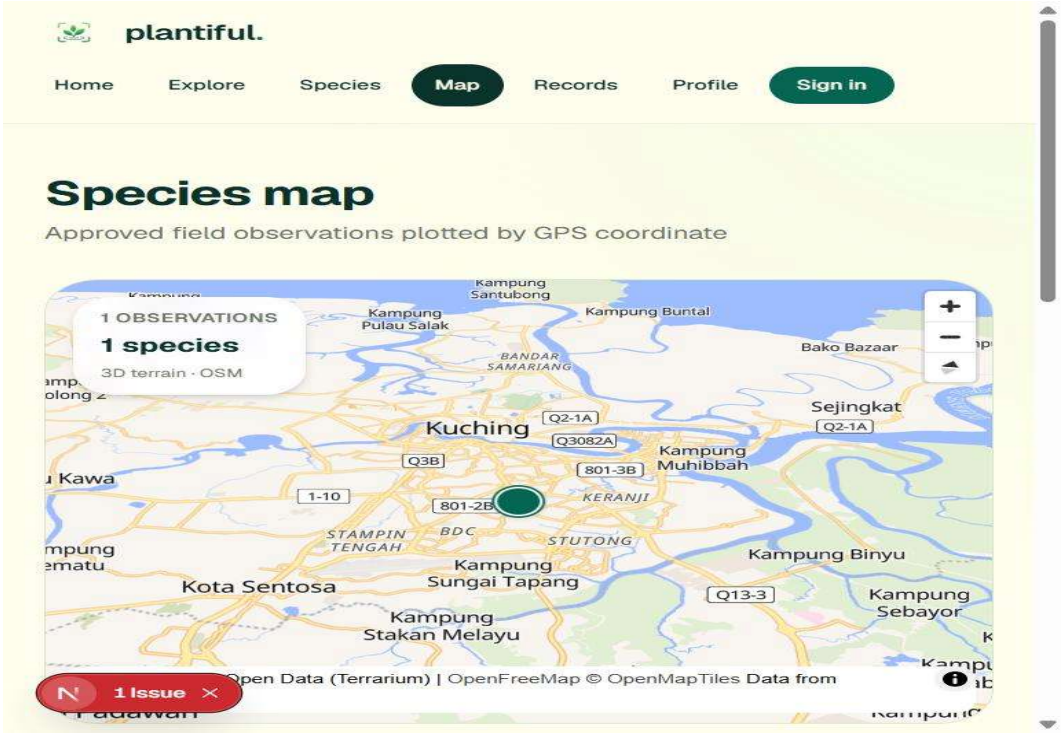
1. Home



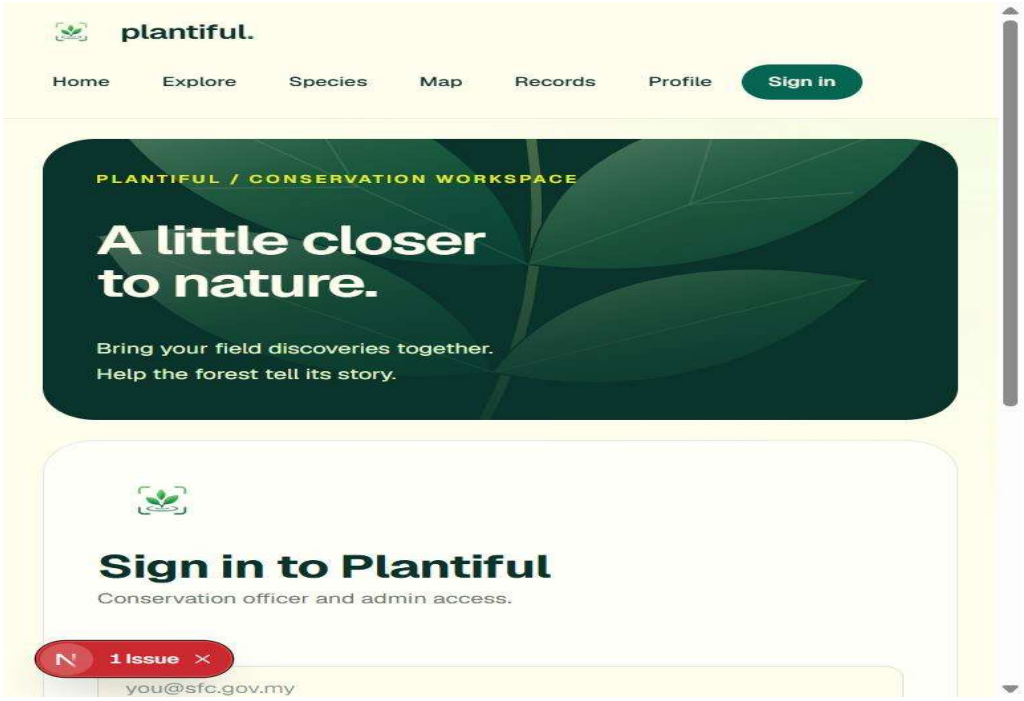

2. Explore



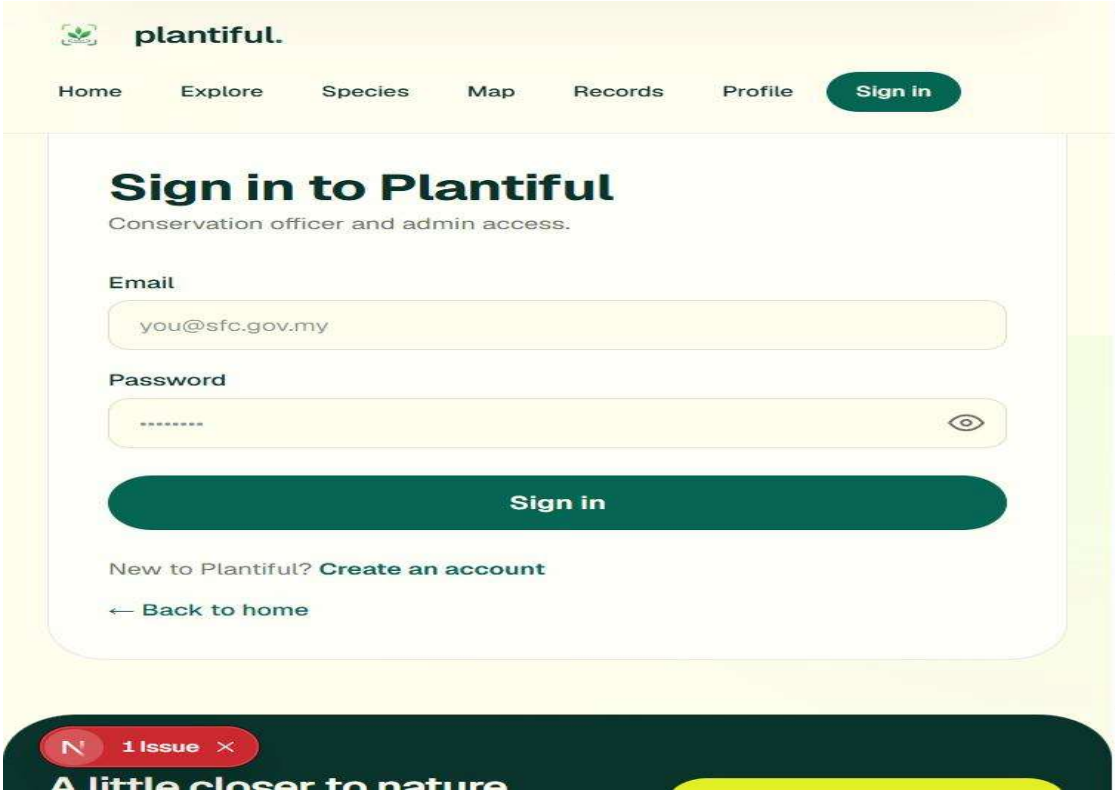

3. Map



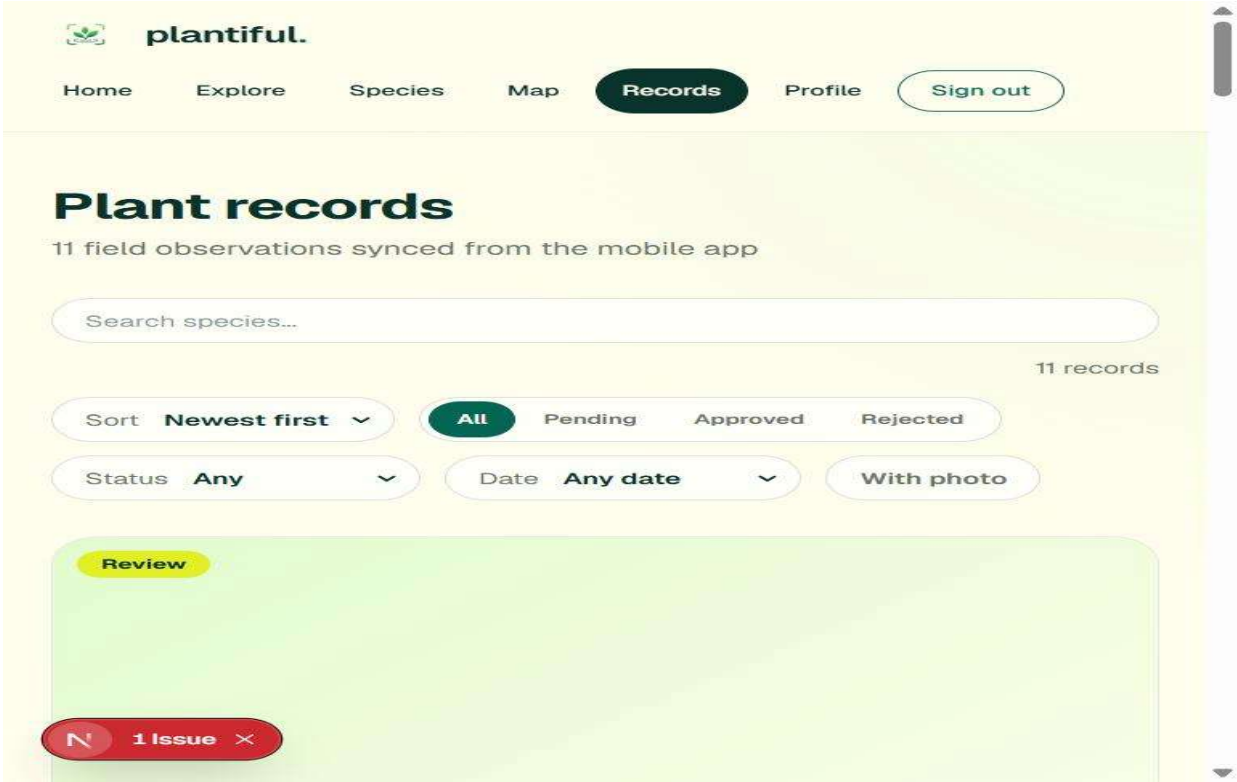

4. Sign in



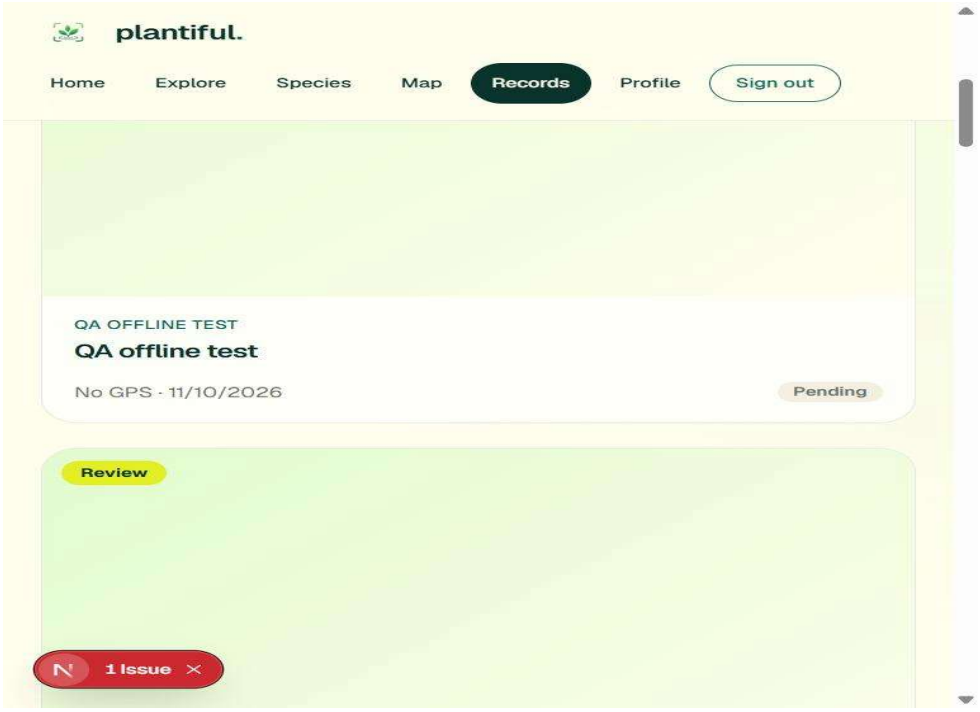

5. Sign In Form



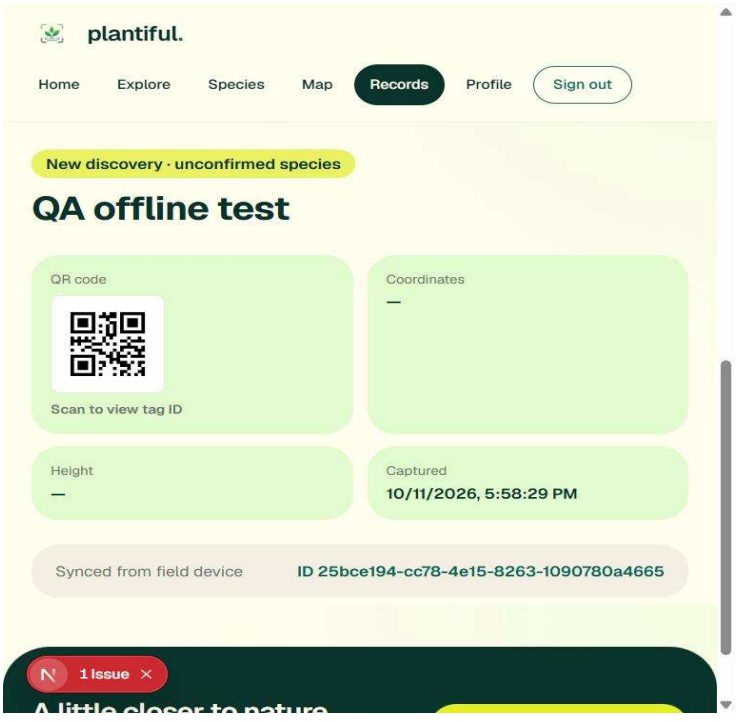

6. Records List



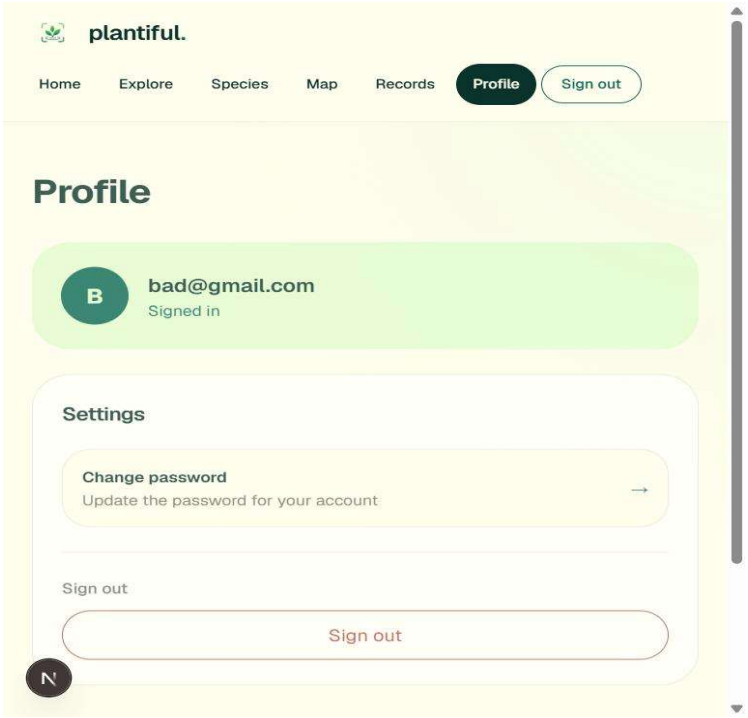
7. Records card



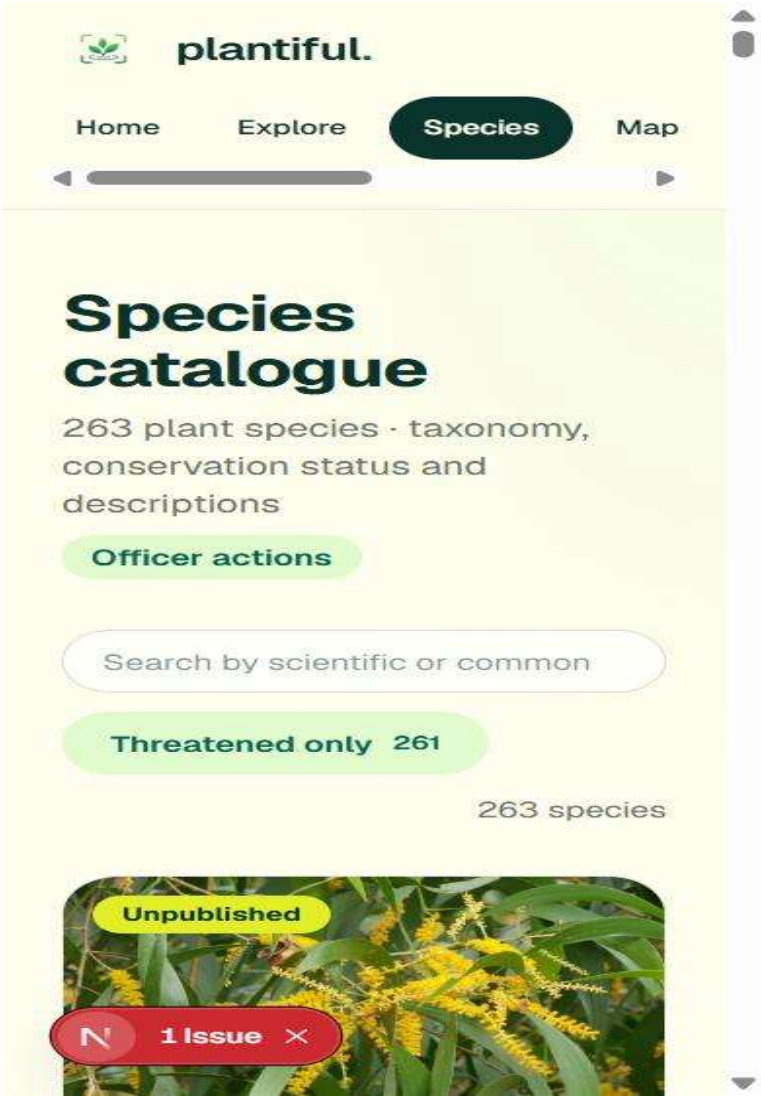

8. Records detail



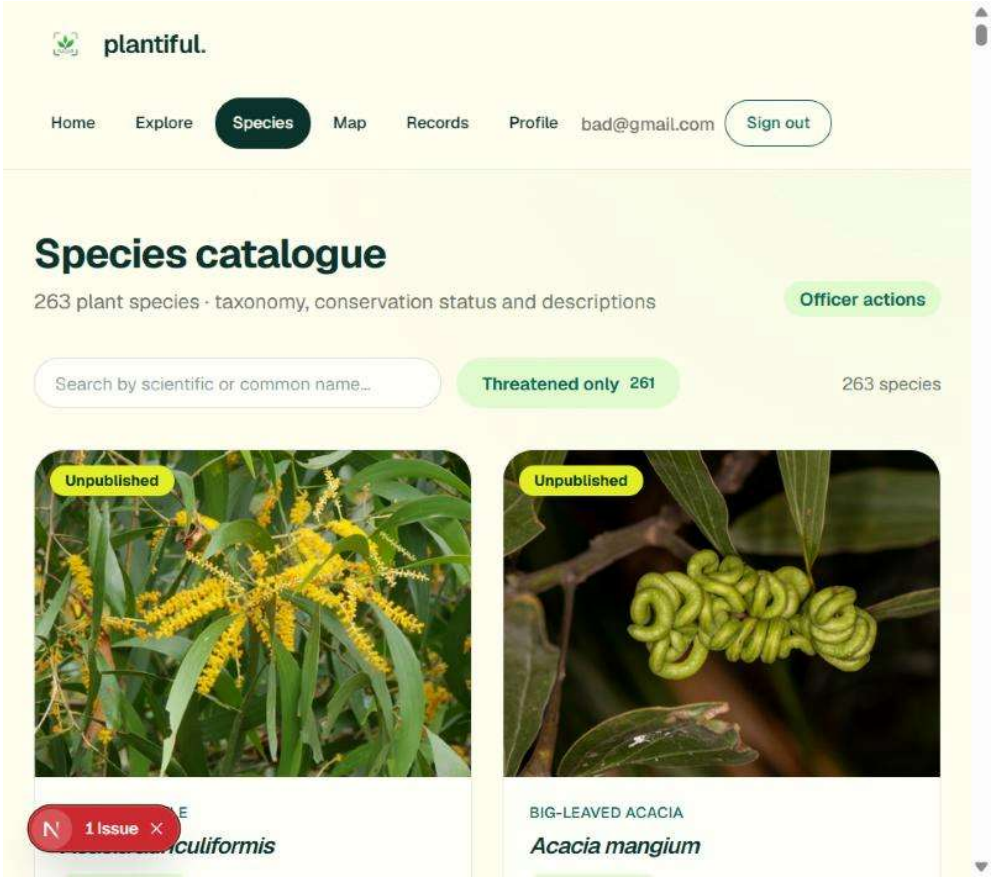
9. Profile


plantiful.
Home
Explore
Species
Map
Records
Profile
Sign out
Profile
B
bad@gmail.com
Signed in
Settings
Change password
Update the password for your account
Sign out
Sign out

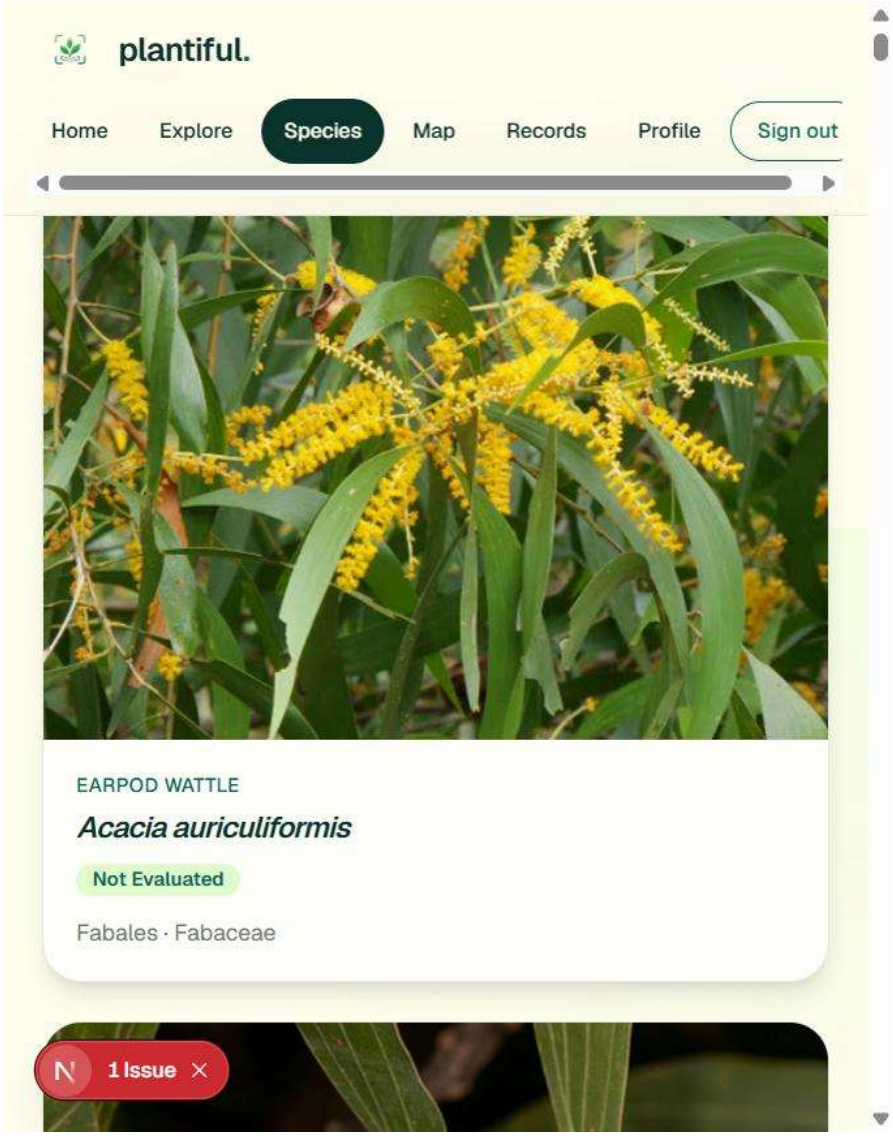
10. Species List

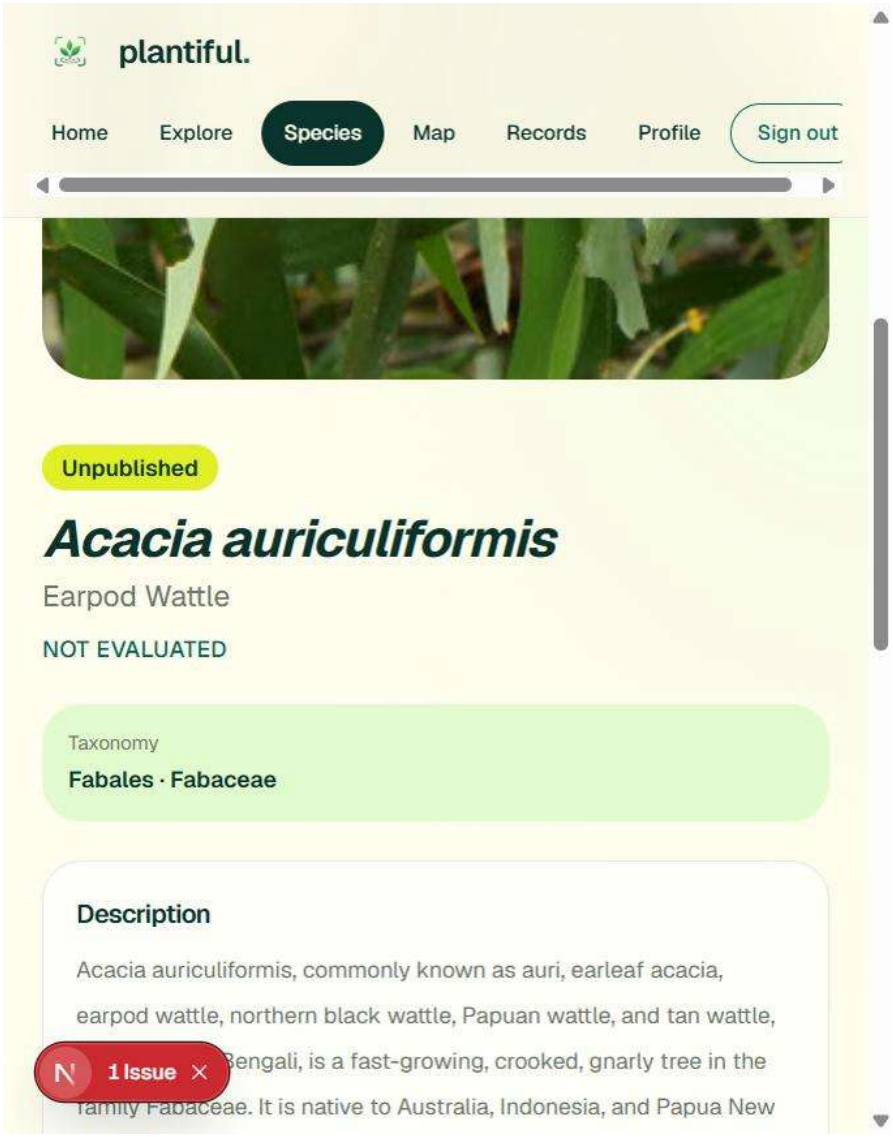



11. Species Catalog

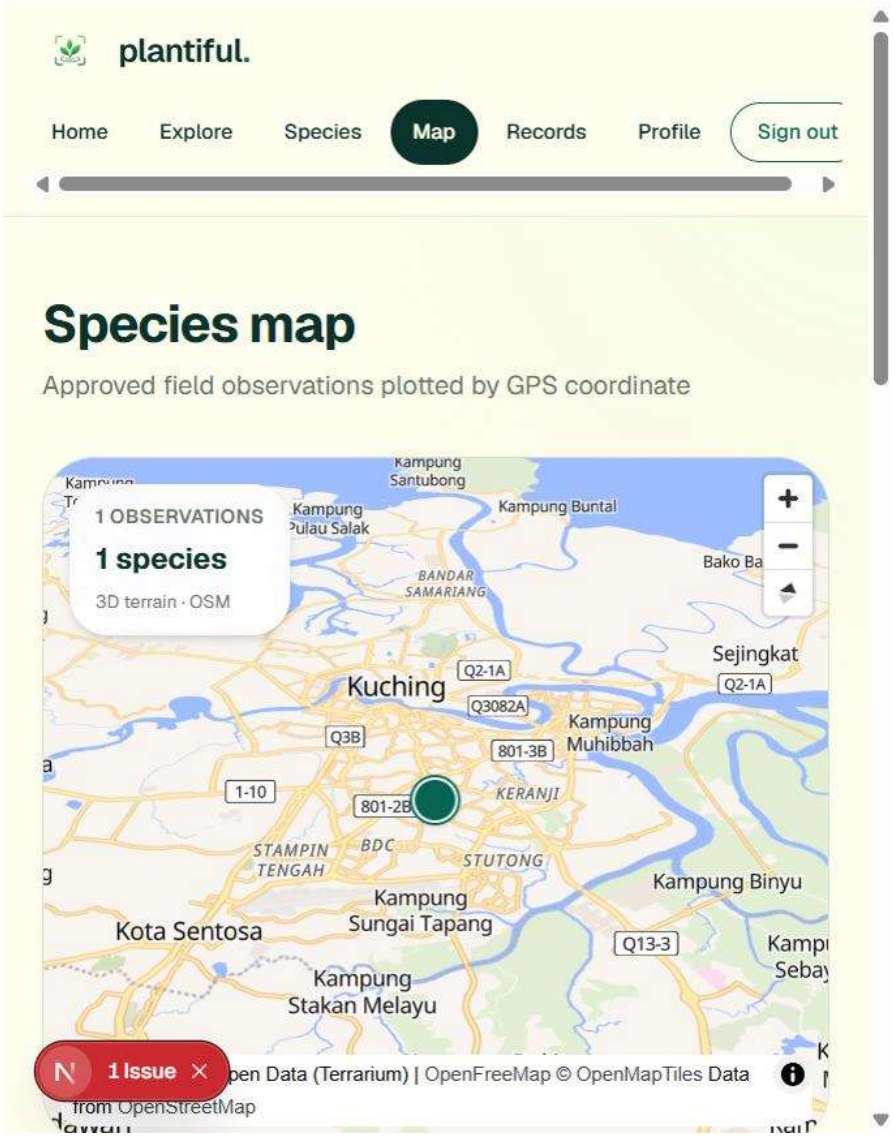



12. Species Deltails


plantiful.
Home
Explore
Species
Map
Records
Profile
Sign out
EARPOD WATTLE
Acacia auriculiformis
Not Evaluated
Fabales · Fabaceae
1 Issue


plantiful.
Home
Explore
Species
Map
Records
Profile
Sign out
Unpublished
Acacia auriculiformis
Earpod Wattle
NOT EVALUATED
Taxonomy
Fabales · Fabaceae
Description
Acacia auriculiformis, commonly known as auri, earleaf acacia,
earpod wattle, northern black wattle, Papuan wattle, and tan wattle,
Bengali, is a fast-growing, crooked, gnarly tree in the
family Fabaceae. It is native to Australia, Indonesia, and Papua New
1 Issue

13. Map Live


plantiful.
Home
Explore
Species
Map
Records
Profile
Sign out
Species map
Approved field observations plotted by GPS coordinate
1 OBSERVATIONS
1 species
3D terrain · OSM
Kampung Santubong
Kampung Buntal
Bako Ba
BANDAR SAMARIANG
Sejingkat
Q2-1A
Kuching
Q3082A
Q3B
Kampung Muhibbah
801-3B
1-10
801-2B
KERANJI
STAMPIN TENGAH
BDC
STUTONG
Kampung Binyu
Kampung Sungai Tapang
Kota Sentosa
Q13-3
Kampung Stakan Melayu
1 Issue
pen Data (Terrarium) | OpenFreeMap © OpenMapTiles Data
from OpenStreetMap